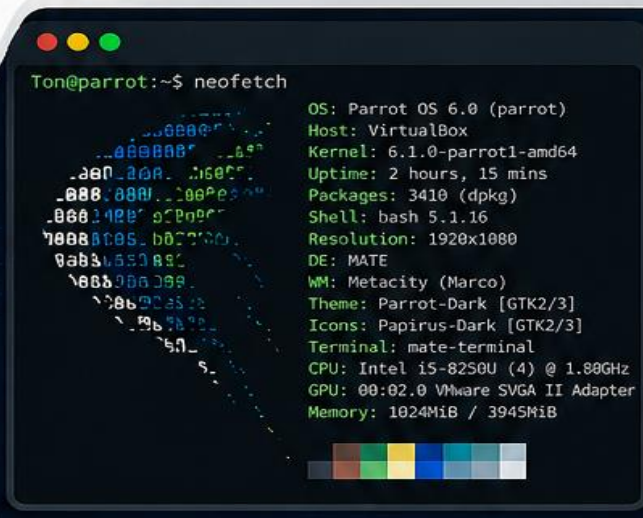
BAHIR DAR UNIVERSITY

Bahir Dar, Ethiopia

OPERATING SYSTEMS ASSIGNMENT

Installation, Viriualization, and UNIX Standardization



Student Name	: Yohannes Damtie Aleka
Student ID	: 1602321
Department	: Software Engineering
University	: Bahir Dar University
Country	: Ethiopia
Course	: Operating Systems
Assignment Type	: Individual Assignment
Operating System Used	: Parrot OS
Academic Year	: 2025
Instructor	: Mr. Wendimu

"The best way to predict the future is to invent it.
— Alan Kay

Building rogust systems
Securing the digital world.

Table of Contents

1. Operating System Installation (Setup) of Parrot OS 2
1.1 Introduction 2
2. Virtualization in Modern Operating Systems 11
2.1 What is Virtualization? 11
3. OS Verification Proof 13
3.1 Command 1: Check OS Release Information 13
4. System Call Demonstration: sysinfo() 14
4.1 Step 1: Writing the C Program 14
References 16

1. Operating System Installation (Setup) of Parrot OS

1.1 Introduction

1.1.1 Background

Parrot OS is a security-oriented Linux distribution based on Debian, meticulously designed for penetration testing, vulnerability assessment, computer forensics, and anonymous web browsing. Developed by the Parrot Project, it provides a comprehensive suite of tools for cybersecurity professionals and enthusiasts. Unlike general-purpose distributions, Parrot OS is pre-configured with a hardened kernel and specialized security tools, making it a robust platform for security-related tasks. It offers multiple editions, including the Security Edition, Home Edition, and Architect Edition, catering to different user needs from full-scale penetration testing to daily privacy-focused computing.

1.1.2 Motivation

The motivation behind my project stems from the need to gain practical, hands-on experience with operating system installation, configuration, and management. For a software engineering student like me, understanding how operating systems interact with hardware and how to effectively utilize a powerful distribution like Parrot OS is crucial. Exploring Parrot OS specifically provides a unique opportunity to engage with advanced security tools and privacy-enhancing technologies, which are vital in the modern digital landscape. My project also aims to bridge the gap between theoretical knowledge of OS concepts and their practical application in a real-world scenario.

1.2 Objectives

The primary objectives of this assignment are as follows:

- **Practical Deployment:** Basically, to successfully document the setup of Parrot OS on physical hardware, ensuring all steps from ISO acquisition to post-installation configuration are covered.
- **Problem Solving:** To identify, document, and provide solutions for technical challenges encountered during the setup and configuration process.
- **System Analysis:** To analyze the filesystem support and architectural advantages of Parrot OS compared to other distributions.

- **Theoretical Understanding:** To provide a deep dive into the concepts of virtualization, explaining its mechanisms and importance in modern IT infrastructure.
- **Standardization Research:** To explore the history and significance of UNIX standardization, including POSIX and other key implementations.

1.3 Hardware & Software Requirements

1.3.1 Hardware Requirements

The following hardware resources are recommended for a smooth experience with Parrot OS on physical hardware:

- **Processor (CPU):** Modern multi-core processor (e.g., Intel Core i3/i5/i7 or AMD Ryzen 3/5/7).
- **RAM:** Minimum 2GB, 4GB or more recommended for optimal performance.
- **Hard Disk Space:** Minimum 20GB, 40GB or more recommended for full installation and tools.
- **Graphics:** Any modern graphics card with support for desktop environments.
- **Network:** Wired or wireless network adapter for internet access.

And in my case I had them all in the packet so it wasn't hard at all.

1.3.2 Software Requirements

The following software components are necessary for the project:

- **Operating System:** Parrot OS (latest stable version, e.g., 6.4 Lorikeet).
- **ISO Image:** Official Parrot OS ISO image (Security Edition recommended).
- **USB Creator:** Tool to create a bootable USB drive (e.g., Etcher, Rufus).

This is where it gets a bit technical in which we will be covering in detail by the installation step.

1.4 Installation process

This section outlines the detailed steps for setting up Parrot OS directly on a physical system. Note that VMware/VirtualBox installation procedures are explicitly excluded as per the assignment requirements.

1.4.1 Step 1 Downloading Parrot OS ISO

The process begins by visiting the official Parrot OS website (parrotsec.org). I Navigated to the download section and selected the “Security Edition.” That edition was the most comprehensive, containing all the tools necessary for penetration testing and security auditing. I Choose the **64-bit ISO image** and verified its integrity using the provided **SHA256 checksum** to ensure the file was not corrupted during download.

1.4.2 Step 2 Creating Bootable USB Drive

After downloading the ISO, I used a tool called Etcher / Rufus to create a bootable USB drive. This involves selecting the downloaded ISO image and the target USB drive, then writing the image to the drive. I ensured all data on the USB drive is backed up as this process will erase it.

1.4.3 Step 3 Booting from USB and Live Environment

I Inserted the bootable USB drive into the target system and restart the computer. Accessed the BIOS/UEFI settings (by pressing F2) and configured the system to boot from the USB drive. Upon one successful boot, the Parrot OS boot menu appeared. I Selected the “Try/Install” option, which boots the system into a live environment. This allows verification that the hardware (network, sound, and display) is correctly recognized before proceeding with the actual installation. The live environment is a fully functional version of Parrot OS running entirely in RAM.

1.4.4 Step 4 Launching Installer and Initial Configuration

From the live desktop, I launched the “Install Parrot” application. Followed the on-screen prompts to select my preferred language (English), time zone (East Africa), and keyboard layout.

1.4.5 Step 5: Partitioning and Installation

This was a critical step. For a fresh installation, it is often easiest to choose the “Erase disk” option, which automatically creates a swap partition and a primary partition formatted with the ext4 filesystem. This is the most reliable method for a dedicated installation. Alternatively, advanced users can manually partition the disk to create custom layouts. And that was the method I used to avoid the disk from being erased. The installer then began copying system files to the hard drive.

1.4.6 Step 6: User Configuration

During the installation, I was prompted to set up the primary user account. As per the assignment requirements, I set the user’s name to my own name **Yohannes Damtie Aleka**. Then chose a secured password and set a hostname for the machine (parrot-yohannes). Finally, I confirmed the installation of the GRUB bootloader to the master boot record of the hard drive. Once the process finished, I rebooted the system and logged into the newly installed Parrot OS. Tada it was great telling the truth.

1.5 Problems I Faced and the respective Solutions.

Even with direct installations, certain issues can arise. Below are common problems I faced and the solutions I took.

- **Slow Package Updates:**
  - **Problem:** Running apt update was extremely slow due to distant mirrors.
  - **Solution:** I Used the Parrot Mirror Selector tool to select faster, local mirrors. This significantly improved the speed of downloading security updates and new tools.

1.6 Filesystem Support Discussion

Parrot OS, being built on the Debian foundation, offers extensive support for a wide range of filesystems. The choice of filesystem is critical for performance, data integrity, and compatibility. Below is a comparison of common filesystems:

Filesystem	Developer	Main Use Case	Key Features	Advantages	Disadvantages
NTFS	Microsoft	Windows (Primary)	Journaling, Access Control Lists (ACLs), Compression	Robust, supports large files/volumes, good security features	Complex permissions, can be slower on non-Windows systems, native Linux support is via FUSE (ntfs-3g)
FAT32	Microsoft	Universal/Legacy	Simple structure, wide compatibility	Extremely compatible across various operating systems and devices	4GB file size limit, no journaling (data loss risk), limited security features
exFAT	Microsoft	Flash Drives/SD Cards	Optimized for flash memory, no 4GB file size limit	Good compatibility with Windows, macOS, and Linux; supports larger files than FAT32	Not as robust as NTFS (limited journaling), lacks advanced features like permissions
ext4	Linux Community	Linux (Primary)	Journaling, Extents, Delayed allocation,	Stable, fast, reliable, widely supported in	Can suffer from fragmentation over long

			supports large filesystems	Linux, good performance	periods (though less than ext3), lacks advanced features of Btrfs/ZFS
Btrfs	Oracle/Community	Modern Linux	Copy-on-Write (CoW), Snapshots, RAID, Checksums, Self-healing	Data integrity, easy rollbacks, flexible storage pooling, built-in RAID	Can have performance overhead in certain workloads, still evolving in some areas
ZFS	Sun Microsystems	Solaris/BSD/Linux	RAID-Z, Snapshots, Pooling, Data integrity, Compression, Deduplication	Massive scalability, enterprise-grade data integrity, powerful features	High RAM requirements, complex to manage for beginners, licensing considerations
HFS+	Apple	Legacy macOS	Journaling, Metadata support, Hard links	Standard for older Macs, good for Apple ecosystem	Case-insensitive issues can cause problems, superseded by APFS, limited cross-platform compatibility

APFS	Apple	Modern macOS/iOS	Space sharing, Snapshots, Strong encryption, Fast directory sizing	Optimized for SSDs, efficient storage, robust for Apple devices	Limited compatibility outside Apple ecosystem, complex internal structure

Why Parrot OS Uses ext4

For its primary installation, Parrot OS typically uses **ext4** as the default filesystem. This choice is based on several factors:

- **Maturity and Stability:** ext4 is a highly mature and stable filesystem that has been extensively tested and refined over many years within the Linux ecosystem. This reliability is paramount for an operating system used in critical security operations.
- **Performance:** ext4 offers a good balance of performance for general-purpose use, including read/write speeds that are efficient for a wide range of tasks, from system operations to handling large security tools.
- **Journaling:** As a journaling filesystem, ext4 ensures data integrity by recording changes to a journal before committing them to the main filesystem. This significantly reduces the risk of data corruption in the event of unexpected power loss or system crashes.
- **Widespread Support:** ext4 is universally supported across all Linux distributions and tools, making it easy to manage and troubleshoot.

1.7 Advantages & Disadvantages of Parrot OS

1.7.1 Advantages of Parrot OS

- **Security by Design:** Unlike standard distributions, Parrot OS comes with a hardened kernel and security-focused configurations out of the box, making it inherently more secure for its intended purpose.
- **Vast Tool Library:** It includes over 600 pre-installed tools for various cybersecurity tasks, including information gathering, vulnerability analysis, exploitation, reverse engineering, and digital forensics.
- **Anonymity Features:** The integration of "Anonsurf" allows users to route their entire system traffic through the Tor network with a single click, protecting their identity during sensitive operations and ensuring privacy.
- **Development Ready:** Beyond security, Parrot OS includes a wide range of compilers, Integrated Development Environments (IDEs), and libraries, making it suitable for software development and system programming.
- **Lightweight Desktop:** The MATE desktop environment is highly customizable and consumes very few resources, leaving more system power available for running demanding security tools.

1.7.2 Disadvantages of Parrot OS

- **Steep Learning Curve:** The sheer number of tools and the complexity of the operating system can be overwhelming for users who are not familiar with Linux or cybersecurity concepts.
- **Not for Gaming/Multimedia:** While it can perform these functions, Parrot OS is not optimized for gaming or high-end multimedia production. Drivers for specialized hardware can sometimes be difficult to configure.
- **Potential for Misuse:** The powerful tools included in Parrot OS can be used for illegal activities if the user does not adhere to ethical guidelines and legal frameworks.
- **Update Frequency:** Being based on Debian Testing, updates are frequent and sometimes large, which can be a burden on slower internet connections and occasionally introduce minor instabilities.

1.8 Conclusion and Future Outlook/Recommendation

1.8.1 Conclusion

The direct installation and configuration of Parrot OS on physical hardware have been a highly educational experience. This project demonstrated the robust capabilities of a specialized Linux distribution designed for cybersecurity. By working through the setup steps and understanding the underlying system choices, a deeper understanding of Linux system administration, disk management, and the importance of a secure operating environment has been gained. Parrot OS has proven to be a powerful and feature-rich platform, offering a professional-grade toolkit for anyone interested in cybersecurity and system programming.

1.8.2 Future Outlook and Recommendations

As I continues to use Parrot OS as a primary operating system still now, the following advanced topics and recommendations are pertinent:

- **Automated Scripting:** Developing shell scripts to automate common security audits, system maintenance tasks, and repetitive operations to enhance efficiency.
- **Containerization:** Exploring the use of Docker or Podman within Parrot OS to deploy isolated applications and services, providing further security and portability.
- **Kernel Hardening:** Investigating custom kernel configurations and security modules to further enhance the system's resilience against attacks.
- **Network Forensics:** Utilizing advanced tools like Wireshark, Zeek, and Suricata to analyze network traffic in real-time, identify anomalies, and investigate security incidents.

I have seen lot's of features from this OS but I also await to be amazed with the uncovered features this operating system can provide.

2. Virtualization in Modern Operating Systems

2.1 What is Virtualization?

Virtualization is a technology that allows for the creation of a virtual (rather than actual) version of something, such as an operating system, a server, a storage device, or network resources. In the context of operating systems, it enables a single physical computer to run multiple isolated operating system instances concurrently. Each virtual operating system, known as a virtual machine (VM), operates as an independent entity with its own virtual hardware that is abstracted from the underlying physical hardware.

2.2 Why is it Important?

Virtualization has revolutionized the IT industry by addressing several critical needs:

- **Resource Optimization:** It allows multiple applications and operating systems to share the same physical hardware, significantly increasing resource utilization and reducing the number of physical machines required. This leads to lower hardware costs and reduced power consumption.
- **Isolation and Security:** Each VM is isolated from others, meaning a security breach or system crash in one VM does not affect the others. This provides a secure sandbox environment for testing and running potentially risky applications.
- **Flexibility and Portability:** VMs can be easily moved, copied, and restored across different physical hosts. This flexibility aids in disaster recovery, load balancing, and rapid deployment of new environments.
- **Testing and Development:** Virtual environments are ideal for testing new software, operating system updates, or security tools without risking the stability of the host system. Developers can create multiple test environments on a single machine.
- **Cost Efficiency:** By reducing the need for physical hardware, organizations save on acquisition costs, power, cooling, and physical data center space.

2.3 How it Works

The core of virtualization is the **Hypervisor**, also known as a Virtual Machine Monitor (VMM). The hypervisor is a layer of software that sits between the physical hardware and the virtual machines. It intercepts calls to the hardware from the guest operating systems and manages the allocation

of physical resources (CPU, RAM, storage, network) to each VM. Modern CPUs also include hardware-assisted virtualization features (e.g., Intel VT-x or AMD-V), which allow the hypervisor to offload complex tasks to the processor, greatly improving performance.

2.4 Hypervisors

Hypervisors are categorized into two main types:

- **Type 1 (Bare-metal Hypervisors):** These run directly on the host hardware, without an underlying operating system. They are typically used in data centers and enterprise environments for their high performance, efficiency, and robust security. Examples include VMware ESXi, Microsoft Hyper-V, and Xen.
- **Type 2 (Hosted Hypervisors):** These run as an application on top of a conventional operating system (the host OS). They are commonly used on personal computers for development, testing, or running different operating systems. Examples include VMware Workstation, Oracle VirtualBox, and Parallels Desktop.

2.5 Virtual Machines (VMs)

Virtual Machines are software emulations of physical computers. Each VM includes its own virtual hardware (CPU, memory, hard disk, network interface) and runs its own guest operating system. From the perspective of the guest OS, it is running on dedicated hardware, even though it is sharing resources with other VMs on the same physical machine.

2.6 Resource Allocation

The hypervisor is responsible for managing and allocating physical resources to each virtual machine. This can be done dynamically, where resources are allocated on demand, or statically, where a fixed amount of resources is assigned to each VM. Efficient resource allocation ensures that each VM has sufficient resources to operate effectively without impacting the performance of other VMs or the host system.

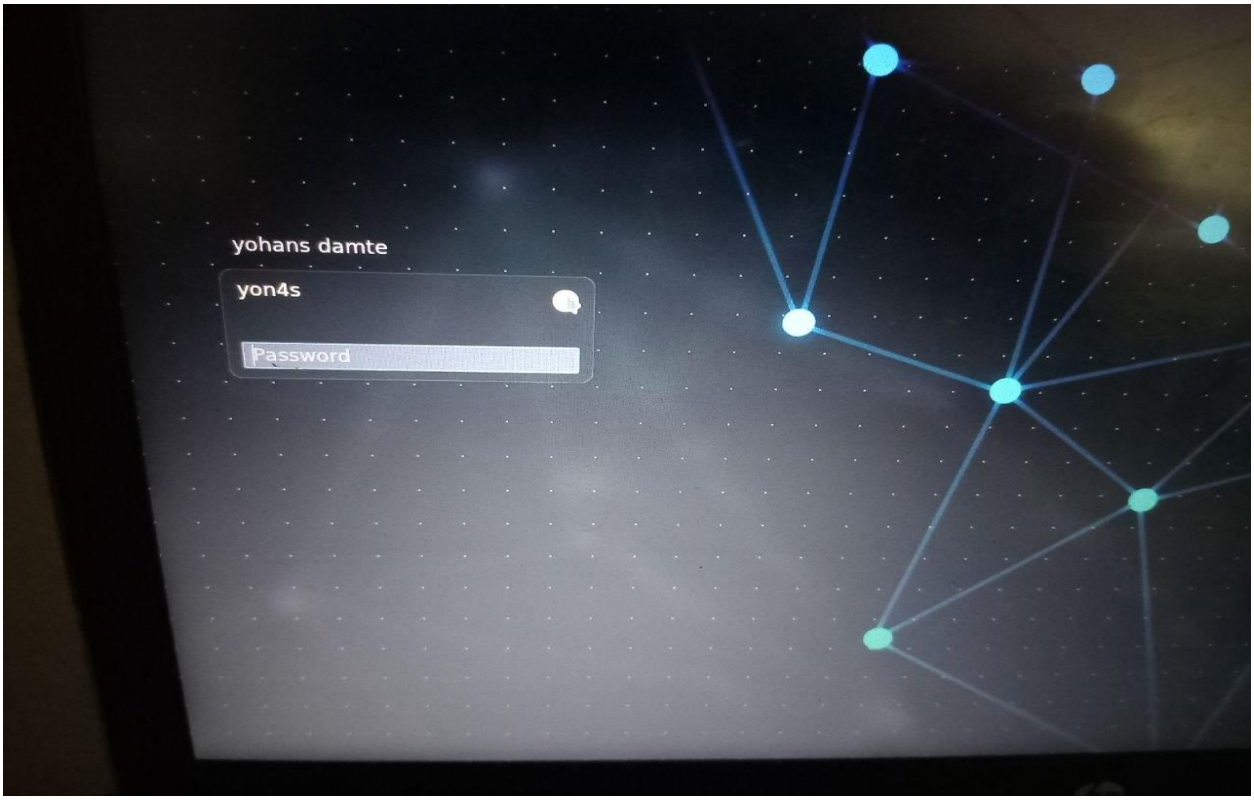
2.7 Security/Isolation

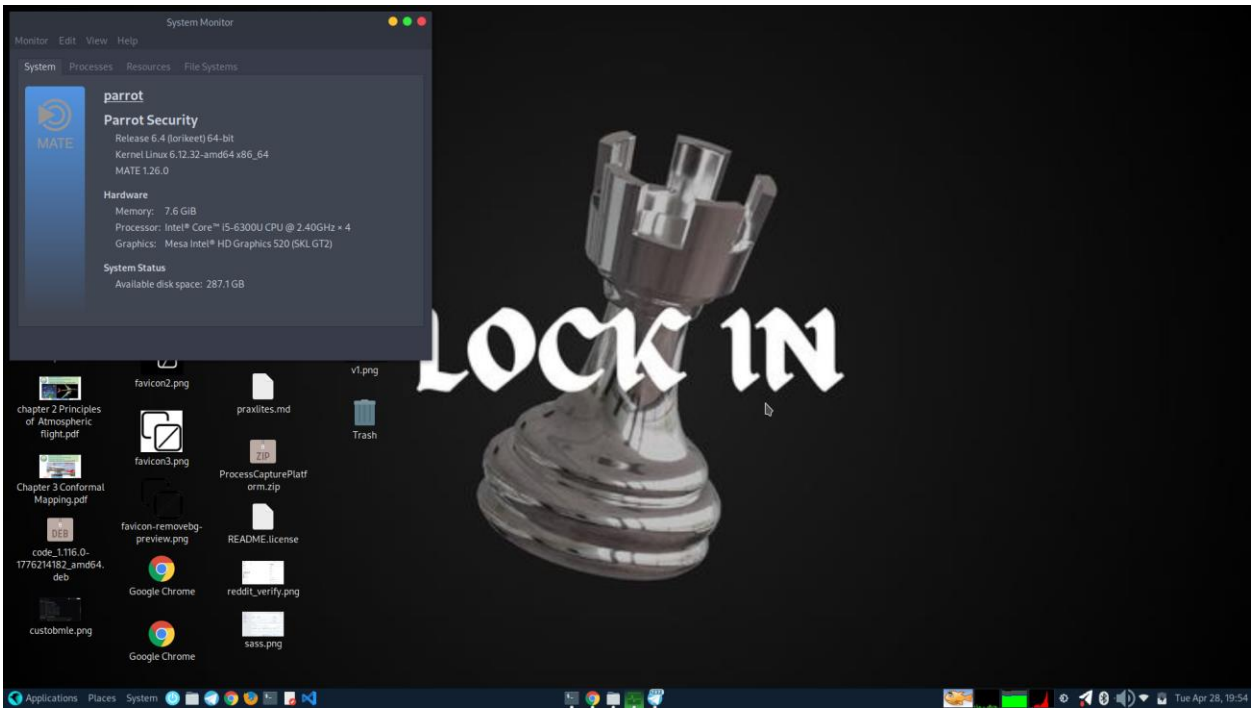
One of the primary benefits of virtualization is the strong isolation it provides. Each VM is a self-contained environment, or sandbox, meaning a security breach or system crash in one VM does not affect the others. This isolation is crucial for maintaining system stability and security, especially in environments where multiple applications or services are running concurrently.

2.8 Common Virtualization Technologies

- **Server Virtualization:** The most common type, allowing multiple virtual servers to run on a single physical server.
- **Network Virtualization:** Combines network resources by splitting the available bandwidth into independent channels, each of which can be assigned to a particular server or device.
- **Storage Virtualization:** Pools physical storage from multiple network storage devices into what appears to be a single storage device that is managed from a central console.
- **Desktop Virtualization:** Separates the desktop environment and applications from the physical client device, allowing users to access their desktops from any location or device.
- **Application Virtualization:** Allows applications to run in an isolated environment from the underlying operating system, preventing conflicts with other applications.

3. OS Verification Proof

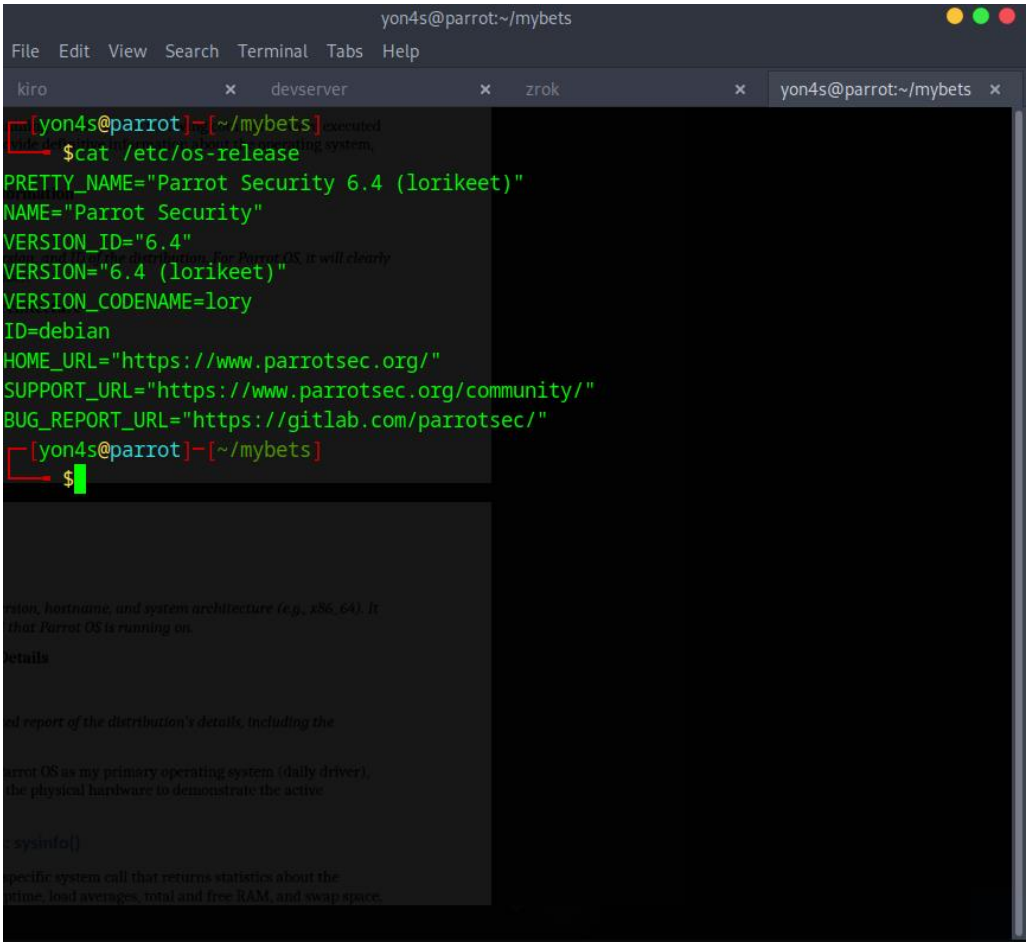




To verify that the system is indeed running Parrot OS, the following commands were executed in the terminal. These commands provide definitive information about the operating system, kernel, and distribution.

3.1 Command 1: Check OS Release Information

cat /etc/os-release

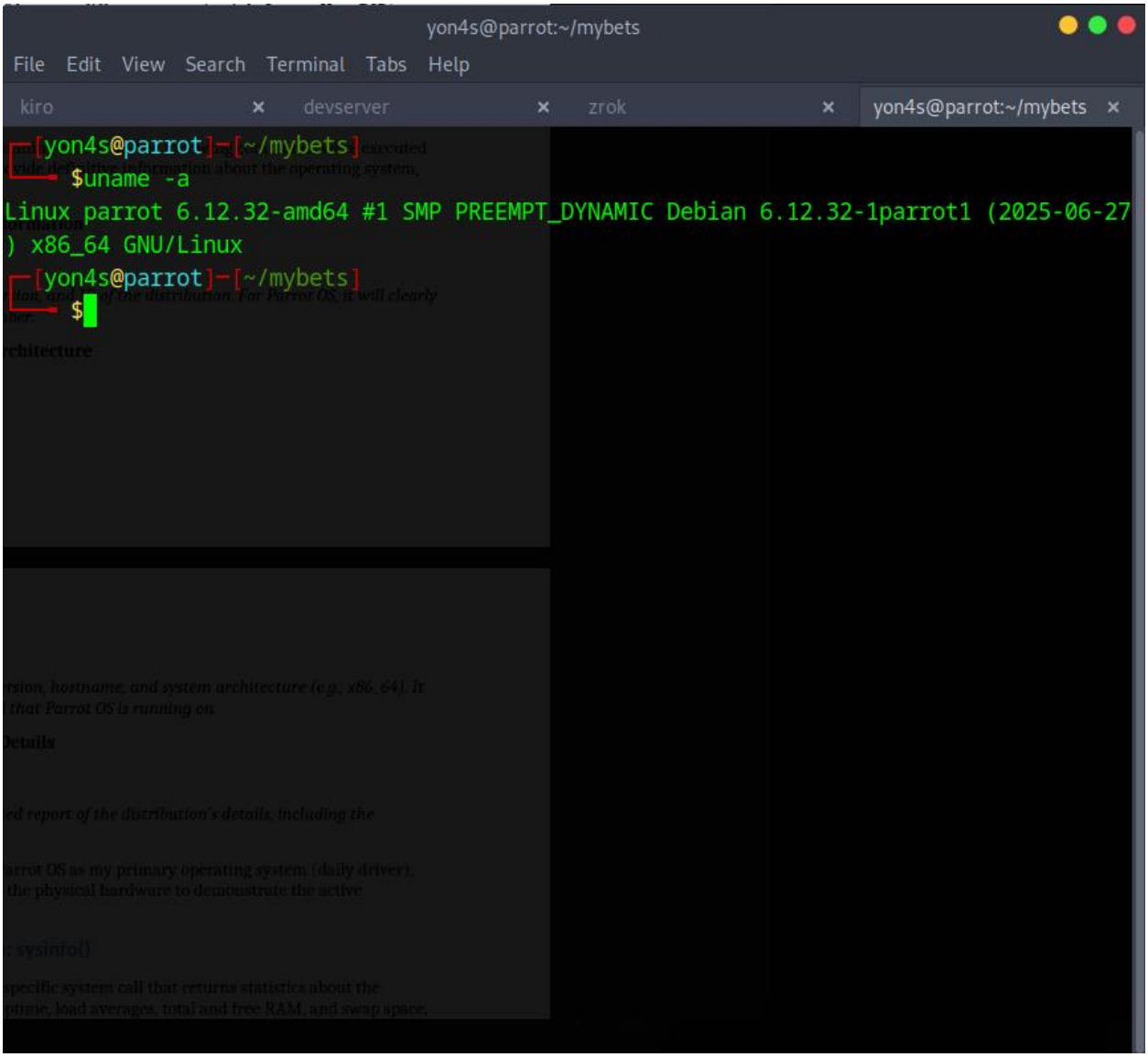


This command displays the name, version, and ID of the distribution. For Parrot OS, it will clearly show “Parrot OS” and its version number.

3.2 Command 2: Check Kernel and Architecture

uname -a

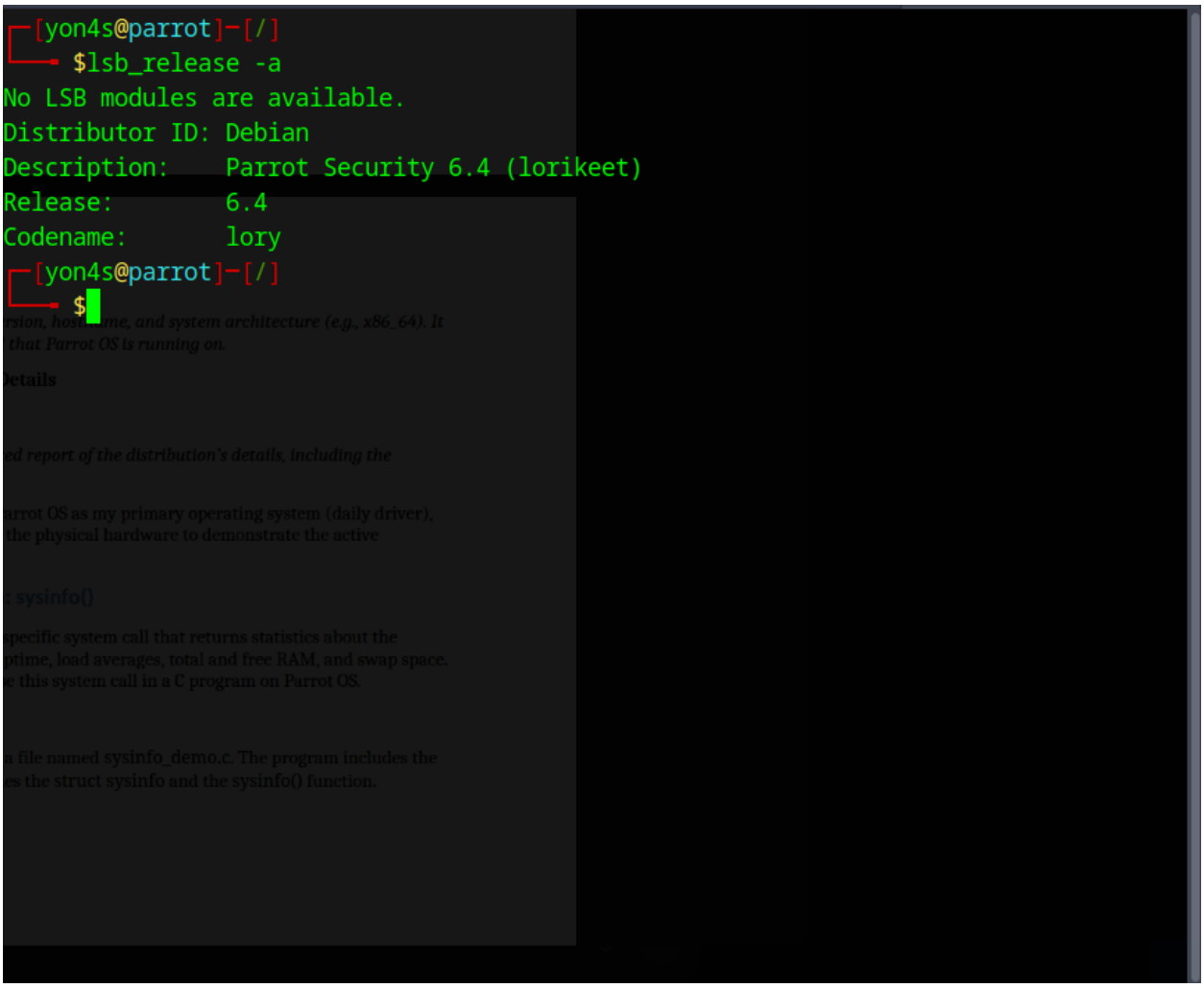
This command provides the kernel version, hostname, and system architecture (e.g., x86_64). It confirms the underlying Linux kernel that Parrot OS is running on.



3.3 Command 3: Check Distribution Details

lsb_release -a

This command provides a standardized report of the distribution's details, including the Distributor ID and Description.

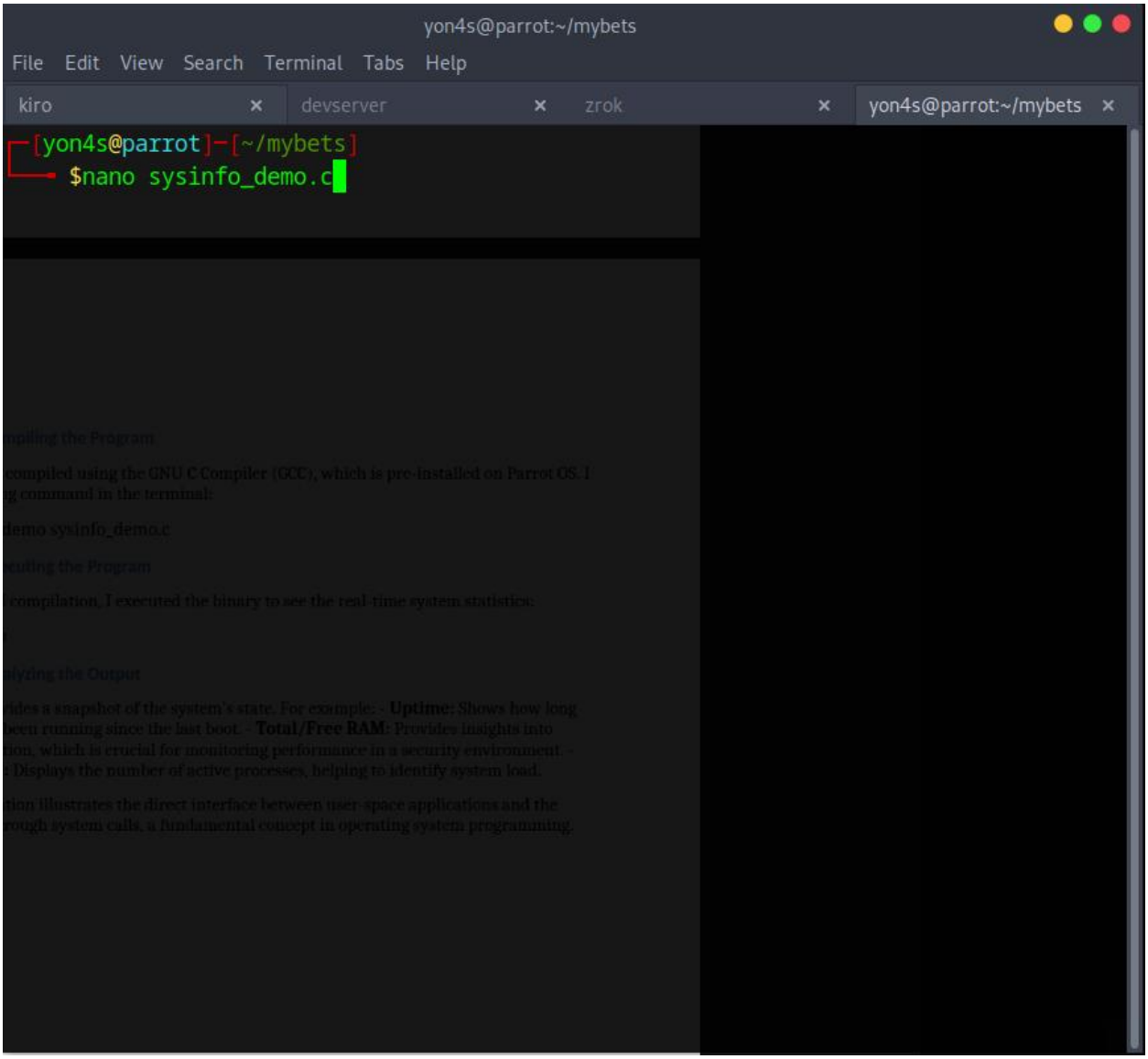


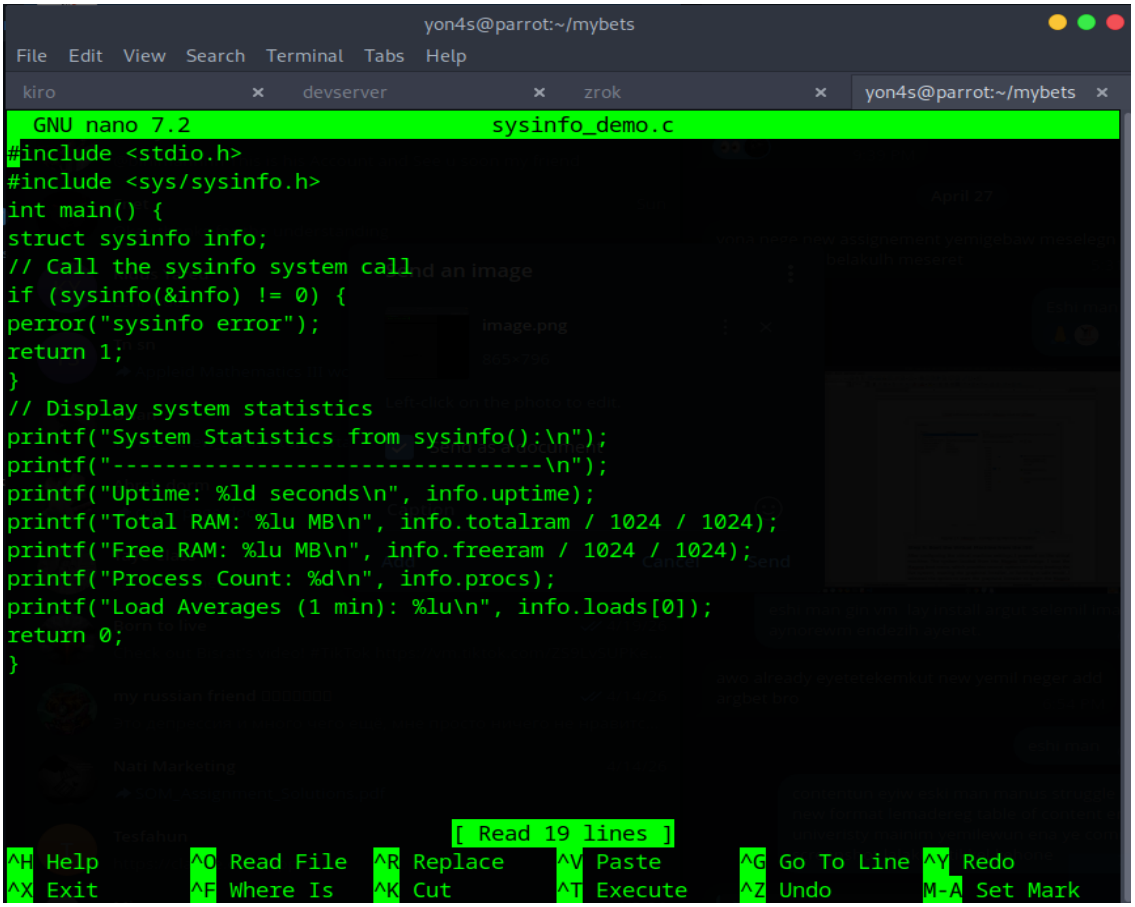
4. System Call Demonstration: sysinfo()

The sysinfo() system call is a Linux-specific system call that returns statistics about the system's resource usage, including uptime, load averages, total and free RAM, and swap space. This demonstration shows how to use this system call in a C program on Parrot OS.

4.1 Step 1: Writing the C Program

I used the nano text editor to create a file named sysinfo_demo.c. The program includes the <sys/sysinfo.h> header, which defines the struct sysinfo and the sysinfo() function.

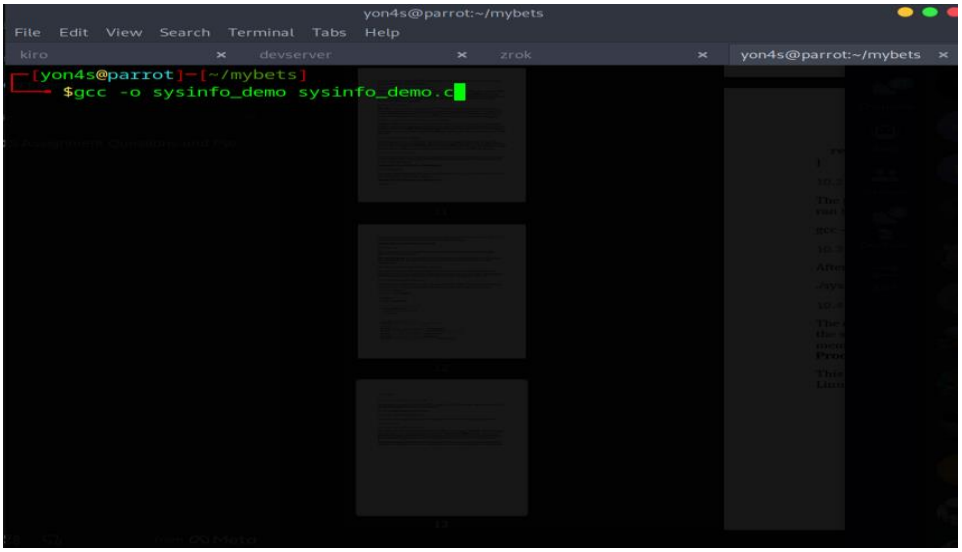




4.2 Step 2: Compiling the Program

The program is compiled using the GNU C Compiler (GCC), which is pre-installed on Parrot OS. I ran the following command in the terminal:

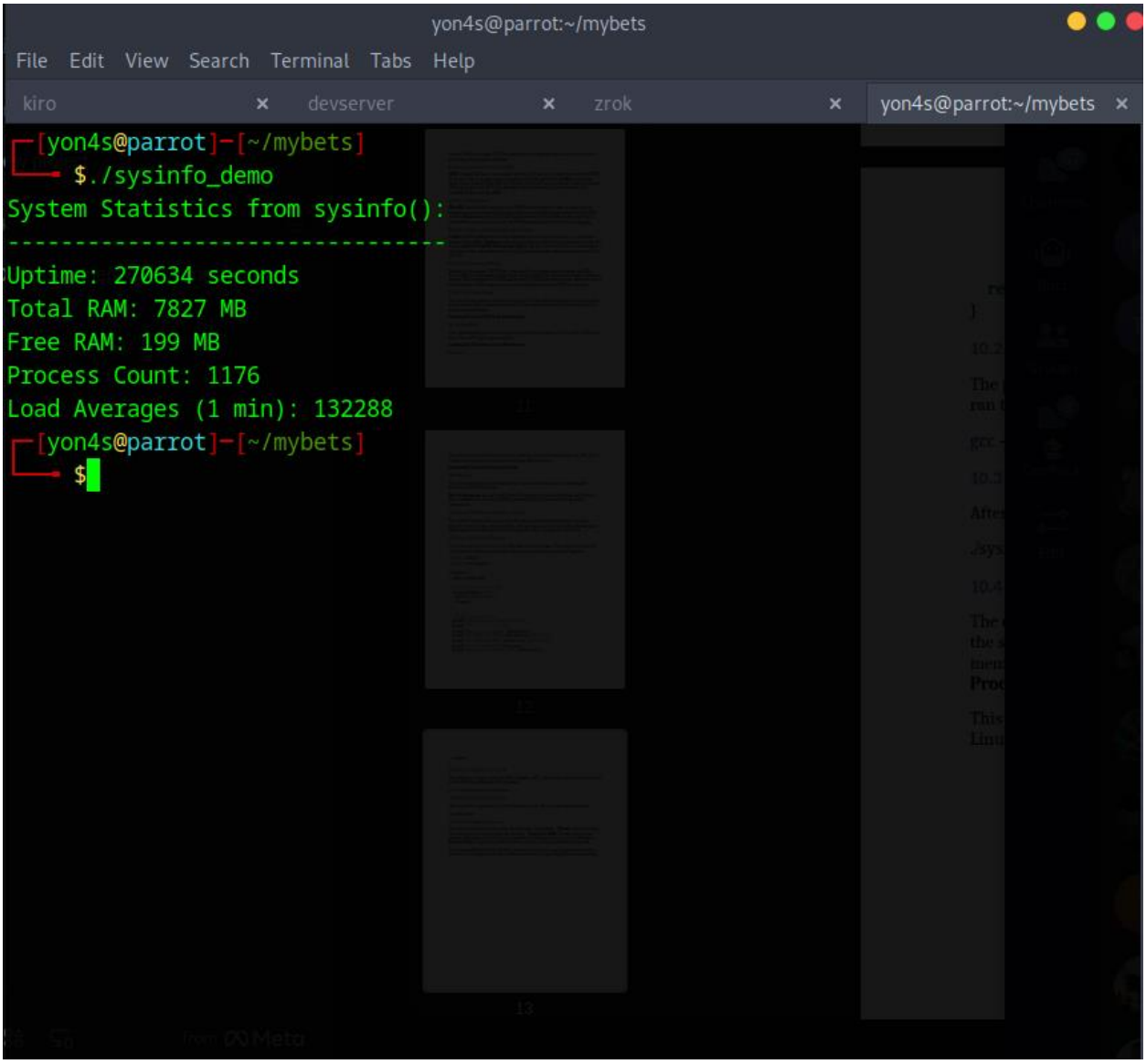
gcc -o sysinfo_demo sysinfo_demo.c



4.3 Step 3: Executing the Program

After successful compilation, I executed the binary to see the real-time system statistics:

./sysinfo_demo



4.4 Step 4: Analyzing the Output

The output provides a snapshot of the system's state. For example:

- **Uptime:** Shows how long the system has been running since the last boot.
- **Total/Free RAM:** Provides insights into memory utilization, which is crucial for monitoring performance in a security environment.
- **Process Count:** Displays the number of active processes, helping to identify system load.
- **Load Averages:** Indicates the average system load over a specific period.

This demonstration illustrates the direct interface between user-space applications and the Linux kernel through system calls, a fundamental concept in operating system programming.

References

[1] Oracle. (n.d.). *What Is Virtualization?* Retrieved from https://www.oracle.com/virtualization/virtualization-explained/

[2] Huntress. (2025, October 3). *What is a Hypervisor and Why It Matters for Cybersecurity*. Retrieved from https://www.huntress.com/cybersecurity-101/topic/what-is-hypervisor

[3] NIST. (2011). *Guide to Security for Full Virtualization Technologies*. Retrieved from https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-125.pdf

[4] IBM. (n.d.). *What Are Hypervisors?*. Retrieved from https://www.ibm.com/think/topics/hypervisors

[5] AWS. (n.d.). *Type 1 vs Type 2 Hypervisors - Difference Between*. Retrieved from https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors/